

UNIVERSITE GASTON BERGER  
DE SAINT-LOUIS

Licence 2 Infographie | UFR CRAC

PROJET DE FIN DE SEMESTRE  
PHP Oriente Objet (POO)

DesignShare — Plateforme de partage de creations  
infographiques

|         |                                               |
|---------|-----------------------------------------------|
| Module  | Programmation Web Avancee — PHP Oriente Objet |
| Duree   | 1 semaine                                     |
| Travail | Soutenance obligatoire                        |
| Rendu   | Archive ZIP + Rapport PDF + Soutenance 15 min |

# 1. Contexte et Description du Projet

Vous etes stagiaire developpeur dans une agence creative senegalaise. L'agence souhaite lancer DesignShare, une plateforme web permettant aux infographistes et etudiants en design de partager leurs creations (affiches, logos, mockups, illustrations), de recevoir des avis de la communaute et de se constituer un portfolio en ligne.

Votre mission est de developper cette plateforme en PHP en appliquant rigoureusement les principes de la Programmation Orientee Objet. Le projet doit demontrer votre maitrise des quatre piliers de la POO : l'encapsulation, l'heritage, le polymorphisme et les interfaces.

## Objectif pedagogique

A l'issue de ce projet, l'etudiant sera capable de concevoir une architecture PHP en classes, d'appliquer l'heritage et les interfaces, de separer la logique metier de la presentation (MVC simplifie), et de manipuler une base de donnees avec PDO.

## 1.1 Nom du projet

**DesignShare** — Plateforme de partage de creations infographiques

## 1.2 Fonctionnalites attendues

- Inscription et connexion des utilisateurs (Etudiant, Designer, Moderateur)
- Publication de creations avec titre, description, categorie et image
- Systeme de commentaires et de reactions (Like, Coup de coeur, Critique constructive)
- Profil utilisateur avec portfolio des creations publiees
- Recherche et filtrage par categorie (Logo, Affiche, Illustration, UI/UX, Motion)
- Espace moderation : signalement et suppression de contenus inappropriés
- Tableau de bord personnel : statistiques de vues et de reactions

## 1.3 Technologies imposees

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| Backend         | PHP 8.x (pur, sans framework) — POO obligatoire        |
| Base de donnees | MySQL 8 via PDO (pas de MySQLi)                        |
| Frontend        | HTML5, CSS3 (Bootstrap 5 autorise), JavaScript vanilla |
| Architecture    | MVC simplifie (classes separees par responsabilite)    |
| Serveur local   | XAMPP ou WAMP (PHP >= 8.0)                             |

## 2. Concepts POO a Implementer

Chaque concept doit etre explicitement visible dans votre code. Le correcteur verifiera leur presence et leur pertinence lors de la soutenance.

### 2.1 Classes & Objets (obligatoire)

Vous devez creer au minimum les classes suivantes, chacune dans son propre fichier PHP :

| Classe      | Attributs principaux                                                  | Methodes principales                     |
|-------------|-----------------------------------------------------------------------|------------------------------------------|
| Utilisateur | id, nom, email, motDePasseHash, role, dateInscription                 | inscrire(), connecter(), getProfile()    |
| Creation    | id, titre, description, cheminImage, categorie, datePublication, vues | publier(), archiver(), incrementerVues() |
| Commentaire | id, contenu, dateCreation, idCreation, idAuteur                       | ajouter(), supprimer(), signaler()       |
| Reaction    | id, type, idCreation, idUtilisateur, date                             | ajouter(), retirer(), compter()          |
| Categorie   | id, libelle, description, icone                                       | listerCreations(), compterCreations()    |
| Database    | instance PDO (singleton)                                              | getInstance(), query(), execute()        |

### 2.2 Encapsulation (obligatoire)

Tous les attributs de vos classes doivent etre prives ou proteges. L'accès se fait exclusivement via des accesseurs (getters/setters) avec validation des donnees.

```
class Creation {
    private int    $id;
    private string $titre;
    private string $categorie;
    private int    $vues = 0;

    public function getTitre(): string { return $this->titre; }

    public function setTitre(string $titre): void {
        if (strlen(trim($titre)) < 3) {
            throw new InvalidArgumentException('Titre trop court.');
```

#### Critere evalue

Chaque setter doit contenir au minimum une validation (longueur, format, plage de valeur). Un setter sans validation ne sera pas credité.

## 2.3 Heritage & Polymorphisme (obligatoire)

La classe Utilisateur est la classe parente. Vous devez creer au moins trois classes filles representant les roles de la plateforme :

```
abstract class Utilisateur {
    protected string $role;
    abstract public function getPermissions(): array;
    abstract public function getDashboard(): string;

    public function __toString(): string {
        return $this->prenom . ' ' . $this->nom . ' (' . $this->role . ')';
    }
}

class EtudiantDesigner extends Utilisateur {
    public function getPermissions(): array {
        return ['publier', 'commenter', 'reagir'];
    }
    public function getDashboard(): string { return 'dashboard/etudiant.php'; }
}

class Designer extends Utilisateur {
    private string $specialite;
    public function getPermissions(): array {
        return ['publier', 'commenter', 'reagir', 'featured'];
    }
    public function getDashboard(): string { return 'dashboard/designer.php'; }
}

class Modérateur extends Utilisateur {
    public function getPermissions(): array {
        return ['publier', 'commenter', 'reagir', 'supprimer', 'bannir'];
    }
    public function getDashboard(): string { return 'dashboard/modérateur.php'; }
}
```

## 2.4 Interfaces & Classes Abstraites (obligatoire)

Vous devez implementer les deux interfaces suivantes dans les classes appropriées :

```
// Interface pour tout contenu publiable sur la plateforme
interface Publiable {
    public function publier(): bool;
    public function archiver(): bool;
    public function estVisible(): bool;
}

// Interface pour tout element pouvant etre signale
interface Signalable {
    public function signaler(string $raison, int $idSignaleur): bool;
    public function getNbSignalements(): int;
    public function estSignale(): bool;
}

// Creation implementera les deux interfaces
class Creation extends Contenu implements Publiable, Signalable {
    // ... implementation de toutes les methodes des interfaces
}
```

---

```
// Commentaire n'implémentera que Signalable  
class Commentaire implements Signalable {  
    // ... implementation de Signalable uniquement  
}
```

## 3. Base de Donnees

### 3.1 Schema SQL fourni (a implementer)

Le schema ci-dessous est fourni a titre indicatif. Vous pouvez l'adapter, mais toutes les tables doivent etre presentes :

```
CREATE TABLE utilisateurs (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  nom         VARCHAR(100) NOT NULL,  
  prenom      VARCHAR(100) NOT NULL,  
  email       VARCHAR(150) UNIQUE NOT NULL,  
  mot_de_passe VARCHAR(255) NOT NULL, -- password_hash()  
  role        ENUM('etudiant','designer','moderateur') DEFAULT 'etudiant',  
  avatar      VARCHAR(255),  
  bio         TEXT,  
  date_inscription DATETIME DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE categories (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  libelle     VARCHAR(80) NOT NULL,  
  icone       VARCHAR(50)  
);  
  
CREATE TABLE creations (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  titre       VARCHAR(200) NOT NULL,  
  description  TEXT,  
  chemin_image VARCHAR(255) NOT NULL,  
  id_categorie INT,  
  id_auteur   INT NOT NULL,  
  vues        INT DEFAULT 0,  
  statut      ENUM('publie','archive','signale') DEFAULT 'publie',  
  date_publication DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_categorie) REFERENCES categories(id),  
  FOREIGN KEY (id_auteur)   REFERENCES utilisateurs(id) ON DELETE CASCADE  
);  
  
CREATE TABLE commentaires (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  contenu     TEXT NOT NULL,  
  id_creation INT NOT NULL,  
  id_auteur   INT NOT NULL,  
  signalements INT DEFAULT 0,  
  date_creation DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_creation) REFERENCES creations(id) ON DELETE CASCADE,  
  FOREIGN KEY (id_auteur)   REFERENCES utilisateurs(id) ON DELETE CASCADE  
);  
  
CREATE TABLE reactions (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  type        ENUM('like','coup_de_coeur','critique') NOT NULL,  
  id_creation INT NOT NULL,  
  id_auteur   INT NOT NULL,  
  date_reaction DATETIME DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE KEY unique_reaction (id_creation, id_auteur),  
  FOREIGN KEY (id_creation) REFERENCES creations(id) ON DELETE CASCADE,  
  FOREIGN KEY (id_auteur)   REFERENCES utilisateurs(id) ON DELETE CASCADE  
);
```

### 3.2 Classe Database (Singleton PDO) — fournie, a completer

```
class Database {
    private static ?Database $instance = null;
    private PDO $pdo;

    private function __construct() {
        $dsn = 'mysql:host=localhost;dbname=designshare;charset=utf8mb4';
        $this->pdo = new PDO($dsn, 'root', '', [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]);
    }

    public static function getInstance(): self {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    public function query(string $sql, array $params = []): PDOStatement {
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);
        return $stmt;
    }
}
```